

IMPLEMENTAÇÃO DE BANCO DE DADOS

Material Técnico de Apoio | 3º Semestre — TADS Rio Grande/RS

Tema: Common Table Expressions (CTE) no PostgreSQL

1. Introdução conceitual

No desenvolvimento de sistemas complexos, é comum nos depararmos com consultas SQL extensas, com múltiplas subconsultas (*subqueries*) aninhadas que prejudicam a legibilidade e a manutenção do código. No PostgreSQL, uma **CTE (Common Table Expression)** resolve esse problema funcionando como um resultado temporário nomeado dentro de uma única instrução (SELECT, INSERT, UPDATE ou DELETE).

Pense na CTE como uma **variável de escopo local para consultas**: ela cria uma 'tabela virtual' na memória apenas durante o tempo de execução daquela query específica, organizando o fluxo de processamento de forma linear (de cima para baixo).

2. Comparativo: Subconsulta vs. CTE

Para entender a diferença de legibilidade, observe os cenários abaixo onde desejamos listar produtos cujo valor é maior do que a média geral de estoque.

Abordagem Tradicional (Subquery)	Abordagem Moderna (Com CTE)
<pre>SELECT nome, preco FROM produtos WHERE preco > (SELECT AVG(preco) FROM produtos);</pre>	<pre>WITH media_estoque AS (SELECT AVG(preco) AS valor_medio FROM produtos) SELECT p.nome, p.preco FROM produtos p CROSS JOIN media_estoque m WHERE p.preco > m.valor_medio;</pre>

3. Estrutura e Sintaxe Padrão

A cláusula obrigatória que inicia uma CTE é a palavra-chave **WITH**. Veja a anatomia da instrução:

```
WITH nome_da_cte AS (
    -- Sua consulta base (Tabela Virtual)
    SELECT coluna1, coluna2
    FROM tabela_real
    WHERE condicao = true
)
-- Consulta Principal que consome a CTE
SELECT *
FROM nome_da_cte;
```

4. Vantagens na Implementação de Sistemas

- **Legibilidade de Código:** Transforma consultas complexas 'de dentro para fora' em sequências lógicas estruturadas de cima para baixo.
- **Reutilização de Blocos:** É possível referenciar a mesma CTE múltiplas vezes na consulta principal (ex: em diferentes JOINS), evitando reescrever o mesmo código.
- **Preparação para Recursividade:** Permite o uso do modificador WITH RECURSIVE, essencial para manipular estruturas hierárquicas (árvores, organogramas ou redes).

5. Estudo de Caso Resolvido

Problema: Em um sistema acadêmico do TADS, precisamos encontrar os alunos matriculados que possuem uma média de notas estritamente maior que a média geral do curso.

```
-- Passo 1: Calcula a média global do curso
WITH media_global AS (
    SELECT AVG(nota) AS nota_media_geral
    FROM matriculas
),
-- Passo 2: Consolida o desempenho individual por aluno
desempenho_alunos AS (
    SELECT aluno_id, AVG(nota) AS media_aluno
    FROM matriculas
    GROUP BY aluno_id
)
-- Passo 3: Filtra e exibe os dados finais cruzando as tabelas virtuais
SELECT a.nome, d.media_aluno, m.nota_media_geral
FROM desempenho_alunos d
JOIN alunos a ON a.id = d.aluno_id
CROSS JOIN media_global m
WHERE d.media_aluno > m.nota_media_geral;
```